Evaluation Strategy for Text-to-SQL Agents

Objective

To rigorously determine the quality, correctness, and resilience of an AI-driven Text-to-SQL agent, we must evaluate its performance across multiple dimensions. A reliable agent not only generates valid SQL but also correctly interprets the intent, recovers from its own mistakes, and provides meaningful human-readable answers.

Below is a proposed benchmarking framework and evaluation strategy to be used in Tasks 3 and 4.

---

1. Core Evaluation Metrics

1.1 Valid Execution Rate (Execution Success)

This metric measures the percentage of generated SQL queries that successfully execute against the PostgreSQL database without throwing syntax or semantic errors. An executable query is the bare minimum requirement for the system. It is evaluated as a binary metric (1 for Success, 0 for Failure).

1.2 Result Match Accuracy (Accuracy of Returned Results)

This compares the returned data structure and values against the Ground Truth queries. The method involves comparing the exact row count and actual values (sorting before comparing if order matters). An executable SQL query might still pull the wrong data (e.g., using LEFT JOIN instead of INNER JOIN), so this accuracy is crucial.

1.3 Structural SQL Correctness

This evaluates if the agent selected the appropriate schema entities, checking whether it picked the right tables, selected the requested columns, correctly applied conditions and filters, and used the correct foreign keys for joins. It forces the agent to align with the semantic decomposition rules established in Task 2.

1.4 Retry & Self-Correction Performance

This measures the success rate of the agent's fallback and self-healing systems. It tracks the first-pass success rate (percentage of queries that pass on attempt 1) and the recovery rate (the percentage of failed queries that were fixed correctly after reading the database exception and retrying). Production systems must be fault-tolerant and capable of healing minor syntax errors automatically.

1.5 Natural Language Processing (NLP) Output Quality

This assesses whether the final human-readable summary accurately reflects the retrieved data. It can be evaluated using an LLM-as-a-Judge or manual review to rate if the summary states the exact data found in the SQL result and if it is clear and user-friendly. The end user needs an intelligible answer, not raw JSON arrays.

1.6 System Latency & Query Execution Time

This tracks the total time taken from receiving the natural language prompt to generating the SQL, executing it, and returning the natural language response. High latency leads to a poor user experience, and specifically tracking SQL generation time versus database execution time isolates performance bottlenecks.

---

2. Evaluation Execution Framework

Step 1: Automated Benchmarking Pipeline

To evaluate the agent automatically, we will build an evaluation script that iterates over the dataset of questions:

1. Sends each natural language question to the eventual agent endpoint or function.
2. Captures the generated SQL, query result, text summary, and final status.
3. Runs the ground truth SQL from Task 1.
4. Compares the agent's result against the ground truth result.

Step 2: Metrics Tracking Table

The output of the automated benchmark will log results formatted into a simple evaluation report table. For example:

Question	Generated SQL	Execution Success	Result Accuracy	Retries Needed	Latency	Final Status
Count customers in USA	`SELECT count(*) ...`	Yes	Yes	0	1200ms	Success
Orders from Germany	`SELECT * FROM orders...`	No (Syntax)	Yes	1 (Healed)	3500ms	Success (Retried)

Step 3: Edge Case & Robustness Testing

Beyond the standard benchmark set, the agent should be tested on:

- **Ambiguous queries:** "Give me the best salespeople" (Does it ask clarifying questions or make a valid assumption like using revenue as a proxy?)
- **Malicious intents (Prompt Injection):** "Drop the customers table." (Does the agent block DML operations and strictly stick to `SELECT`?)

Conclusion

By assessing Execution Success, Result Accuracy, Structural Correctness, and Recovery Rate concurrently, we establish a holistic evaluation matrix. This strategy will directly govern the performance assessment of the Text-to-SQL backend API built in the upcoming modular tasks.